NUMERICAL ANALYSIS
GUIDED LAB 1

ROB TOMPKINS

1. Compute the absolute and relative error for

$$\sqrt{2} \approx 1 + \frac{24}{60} + \frac{51}{60^2} + \frac{10}{60^3}$$

Solution. Let's first compute the left and right hand side of the equation above using x_{approx} to represent the approximate solution (RHS). Using matlab we see that

$$\begin{aligned} x &\approx 1.41421356237, \text{ and} \\ x_{\text{approx}} &= 1.4142129630 \end{aligned}$$

Let ε_{abs} represent the absolute error, and ε_{rel} represent the relative error. Then we have the following:

$$\begin{aligned} \varepsilon_{\text{abs}} &= |\sqrt{2} - x_{\text{approx}}| \\ &= |1.41421356237 - 1.4142129630| = 5.99 \times 10^{-7} \end{aligned}$$

and we also see that:

$$\begin{aligned} \varepsilon_{\text{rel}} &= \frac{\varepsilon_{abs}}{\sqrt{2}} \\ &\approx \frac{5.99 \times 10^{-7}}{1.41421356237} \\ &\approx 4.24 \times 10_{-7} \end{aligned}$$

□

2. The Newton-Raphason method for determining $\sqrt{a}$ follows from repeated iteration of

$$y_{n+1} = \frac{1}{2}\left(y_n + \frac{a}{y_n}\right) \tag{1}$$

Show that $y = \sqrt{a}$ is the fixed point of this sequence.

Proof. To achieve $y = \sqrt{a}$ as a fixed point of our equation we will need to compare the error between the n^{th} term and the $(n+1)^{\text{st}}$ term.

Date: January 23, 2026.

We begin by subtracting $\sqrt{a}$ from both sides of the equation and performing some algebraic manipulation. Consider

$$\begin{aligned} x_{n+1} - \sqrt{a} &= \frac{1}{2}\left(x_n + \frac{a}{x_n}\right) - \sqrt{a} \\ &= \frac{x_n^2 - 2x_n\sqrt{a} + a}{2x_n} \\ &= \frac{(x_n - \sqrt{a})^2}{2x_n} \end{aligned}$$

Accordingly,

$$\begin{aligned} \lim_{n\to\infty} \frac{|x_{n+1} - \sqrt{a}|}{|x_n - \sqrt{a}} &= \lim_{n\to\infty} \frac{1}{2x_n} \\ &= \frac{1}{2\sqrt{a}} \end{aligned}$$

Hence the sequence generated by this scheme has order of convergence equal to 2 and asymptotic error constant $1/(2\sqrt{a})$. □

3. Create a script that implements Heron's algorithm (the formula in problem 1). It should take as the input: (1) the number a to square root, (2) the initial guess y_0, and (3) the total number N of iterations. In addition to outputting the approximate square root at each iteration. you may use a bulit-in functionas the "exact" answer for comparison. Plot bothe the relative error as a function of iteration for $N = 6$ on a log-log scale for $a = 2$. How sensitive is the result to y our initial guess of y_0?

Solution. First note the file problem3.m.

Consider a function that we have in file "problem3.m" that we have below:

```
function [p,absError,relError] = fixpt(a,y,n)
% Input - a, the number to sqaure root
%       - y, the initial guess
%       - n, the number of iterations to run
    p = y; % Initialize p with the initial guess
    absError = zeros(1,n);
    relError = zeros(1,n);
    for k = 1:n
        p_new = 0.5 * (p + a / p);
        absError(k) = abs(p_new - sqrt(a));
        relError(k) = absError(k) / abs(p_new);
        if absError < 1e-10
            break;
```

```
        end
        p = p_new;
    end
end

format long
[p,absError,relError] = fixpt(2,30,6)
loglog(1:6,absError, '-k')
hold on
loglog(1:6,relError,'--k')
hold off
legend("Absolute Error", "Relative Error")
```

So we see our plot from the above results as: But, if we fiddle with the initial value for the

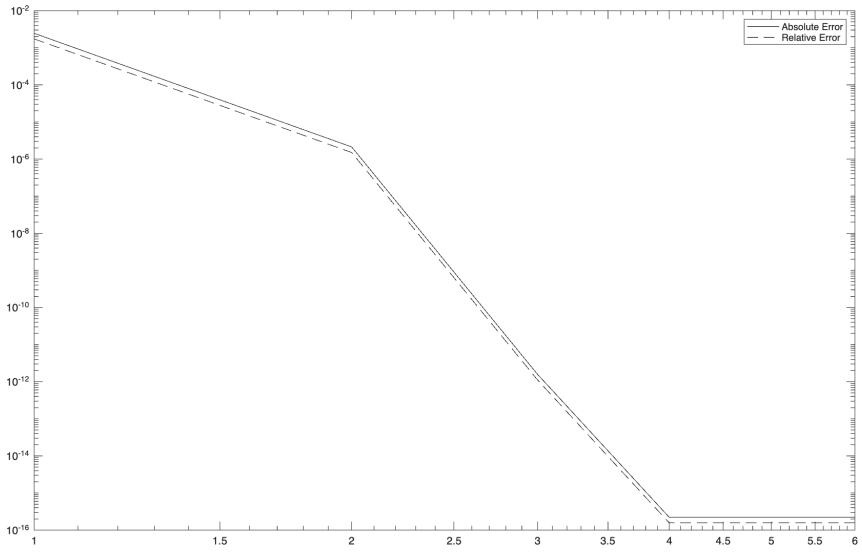

FIGURE 1. Log-Log plot of the absolute and relative error for Newton Raphson for $N = 6$, $y_0 = 1.5$

answer we see different errors. Consider Figure 2 which shows the errors when we set $y_0 = 30$ and we are done. □

4.

Solution. First note the file problem4.m.

Compute the *appoximate* relative error, and compare to the "true" errors. So we vary our algorithm a little as follows

```
function [p,absError,relError] = fixpt(a,y,n)
```

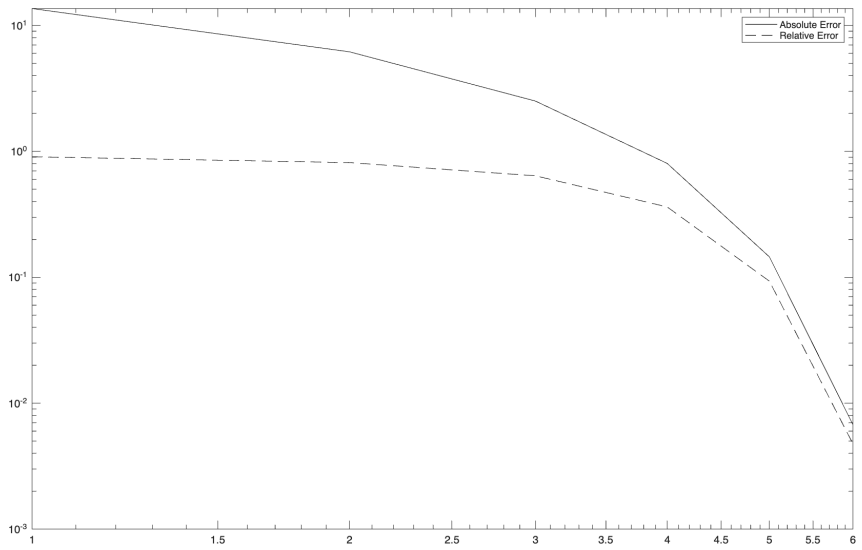

FIGURE 2. Log-Log plot of the absolute and relative error for Newton Raphson for $N = 6$, $y_0 = 30$

```
% Input - a, the number to sqaure root
%       - y, the initial guess
%       - n, the number of iterations to run
    p = y; % Initialize p with the initial guess
    absError = zeros(1,n);
    relError = zeros(1,n);
    for k = 1:n
        p_new = 0.5 * (p + a / p);
        absError(k) = abs(p_new - p);
        relError(k) = absError(k) / abs(p_new);
        if absError < 1e-10
            break;
        end
        p = p_new;
    end
end

format long
[p,absError,relError] = fixpt(2,30,6)
loglog(1:6,absError, '-k')
hold on
loglog(1:6,relError,'--k')
hold off
```

```
legend("Absolute Error", "Relative Error")
```

And for $N = 6$ and $y_0 = 30$ we see get the following errors

$$\varepsilon_6^{\text{abs}} = \begin{bmatrix} 14.966666666666667 \\ 7.450147819660015 \\ 3.659722054452168 \\ 1.706854881541622 \\ 0.657164646924791 \\ 0.138467746307099 \end{bmatrix}$$

and

$$\varepsilon_6^{\text{rel}} = \begin{bmatrix} 0.995565410199557 \\ 0.982456225846853 \\ 0.932778422047204 \\ 0.770029900059511 \\ 0.421409602464346 \\ 0.097445508110978 \end{bmatrix}$$

if we choose the $y_0 = 1.5$ and $N = 6$ our errors drop considerably to:

$$\varepsilon_6^{\text{abs}} = \begin{bmatrix} 0.344444444444444 \\ 0.040754877014419 \\ 0.000586994346855 \\ 0.000000121821181 \\ 0.000000000000005 \\ 0 \end{bmatrix}$$

and

$$\varepsilon_6^{\text{rel}} = \begin{bmatrix} 0.236641221374046 \\ 0.028806090944516 \\ 0.000415067647425 \\ 0.000000086140583 \\ 0.000000000000004 \\ 0 \end{bmatrix}$$

□

5. Plug in the equation $y_n = \sqrt{a}(1 + \delta_n)$ where $\varepsilon_n^{\text{rel}} = |\delta_n|$. Assuming that $|\delta_n| \ll 1$, simplify that to get that $\delta_{n+1} = C\delta_n^2 + \mathcal{O}(\delta_n^3)$ and specify what value the constant C should take.

Solution. Begin by substituting our δ_n formula into (1). We get the following

$$\sqrt{a}(1+\delta_{n+1^{\text{`}}}) = \frac{1}{2}\left(\sqrt{a}(1+\delta_n) + \frac{a}{\sqrt{a}(1+\delta_n)}\right)$$

Now divide both sides by $\sqrt{a}$, and now notice that

$$\begin{aligned}
1+\delta_{n+1} &= \frac{1}{2}\left(1+\delta_n+\frac{1}{1+\delta_n}\right)\\
&= \frac{1}{2}\left(\frac{(1+\delta_n)^2+1}{1+\delta_n}\right)\\
&= \frac{1}{2}\left(\frac{1+2\delta_n+\delta_n^2+1}{1+\delta_n}\right)\\
&= \frac{2+2\delta_n+\delta_n^2}{2(1+\delta_n)}\\
&= 1+\frac{\delta_n^2}{2(1+\delta_n)}
\end{aligned}$$

now consider the taylor polynomial over

$$\begin{aligned}
(1+\delta_n)^{-1} &= \frac{1}{\delta_n}-\left(\frac{1}{\delta_n}\right)^2+\left(\frac{1}{\delta_n}\right)^3-\cdots\\
&= \frac{1}{\delta_n}-\left(\frac{1}{\delta_n}\right)^2+\mathcal{O}\left(\left(\frac{1}{\delta_n}\right)^3\right)-\cdots
\end{aligned}$$

Because the sequence converges and we know that $|\delta_n| \ll 1$ we can obtain

$$\delta_{n+1} = \frac{\delta_n^2}{2}+\mathcal{O}(\delta_n^3)$$

□

6. Compare your theoretical error estimate to you numerically computed errors. Use your first numerically computed error for δ_0.

Solution. First note the file problem6.m.

If we look at the last problem we see that we have the sequence

$$\delta_{n+1} = \frac{\delta_n^2}{2+\delta_n}$$

So we took that and plugged it into our a convergence iteration in the same fashion as the other problems, except we were concerned with convergence to $\delta_n \to 0$ as $n \to \infty$. And our goal was to base this error on earlier computed errors, so we used the $y_0 = 1.5$ run above and grabbed the the first step in the absolute error there specifically $\epsilon_0^{\text{abs}} = 0.344444444444444$ and plugged that into our formula for theoretical error. The plot that we got follows: and

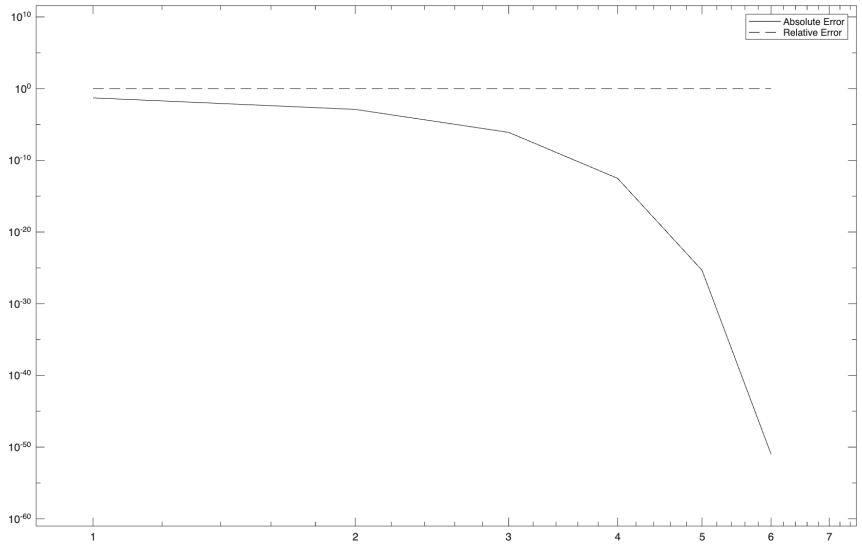

FIGURE 3. Log-Log plot of the theoretical for y_n δ_n in $N = 6$ steps

we are done with this portion of the problem □

7. Modify your script to run for 10 iterations. Notice that the behaviour of the computer error deviates significantly from your theoretical prediction after some number of iterations (how many?). At what value does the numerically computed error seem to level off at all? Why does this happen?

Solution. If we bump up the number of iterations to run in the code from problem 6 from $N = 6$ to $N = 10$, then we get the following plot of error.

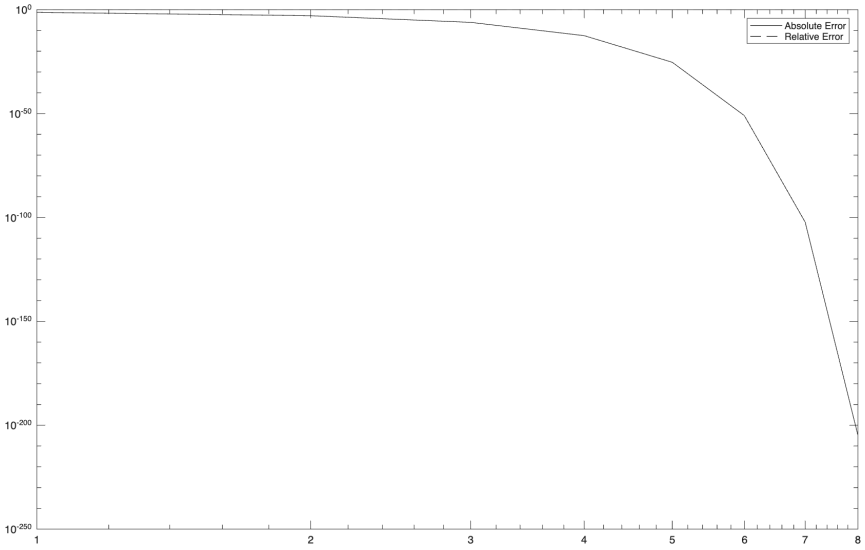

FIGURE 4. Log-Log plot of the theoretical for y_n δ_n in $N = 10$ steps

And we are done with this portion of the problem □

8. Consider the problem $y_n + 1 = y_n + y_n^2 - a$. Show that $y = \sqrt{a}$ is a fixed point of our equation.

Proof. Asuming convergence fixed point c should satisify the equality $c = c + c^2 - a$ so $c = \sqrt{a}$. □

9. Implement the algorithm for the equation in problem 8, and test it with several initial guesses between 1 and 2. What do you notice about how the algorithm's convergence depends on the initial guess.

Solution. First note the file problem9.m.

When we plot the absolute and relative as the series progresses for the different values of $y_0 = 1.2, 1.5, 1.8$ we see vastly differing errors and some that don't converge

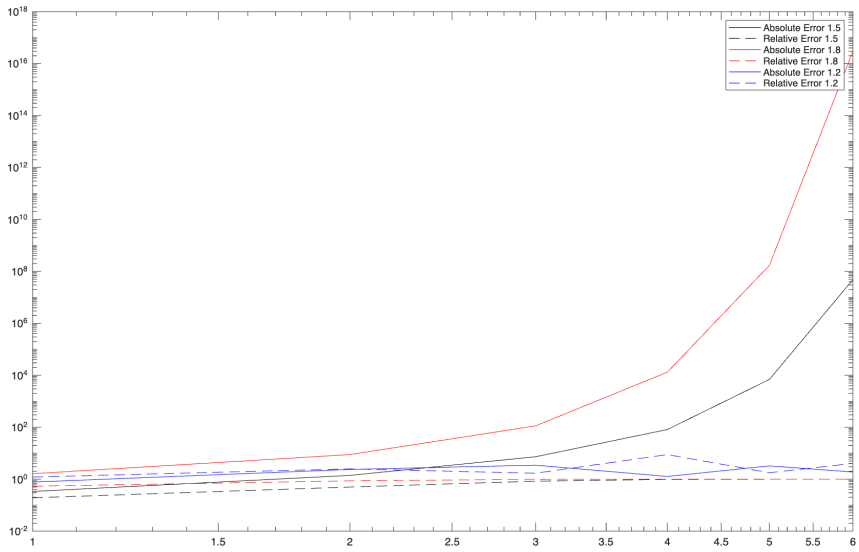

FIGURE 5. Log-Log plot of the absolute and relative error for $y_{n+1} = y_n + y_n^2 - a$ with changes to initial conditions y_0

Curious that for 1.5 and 1.8 we diverge, but for 1.2 we converge. □

10. Explore the error and convergence for other method for computing the square root of two. You can consider the following two iterative methods. Also find (or create) and implem ent one additional method for comparison

$$x_{n+1} = \frac{1}{4}\left(3x_n + \frac{2}{x_n}\right)$$

$$z_{n+1} = \frac{1}{8}\left(3z_n + \frac{12}{z_n} - \frac{4}{z_n^3}\right)$$

Solution. problem10.m.

We went and ran the algorithm for the sequence x_n to see how things would converge. The convergence plot follows ad: □

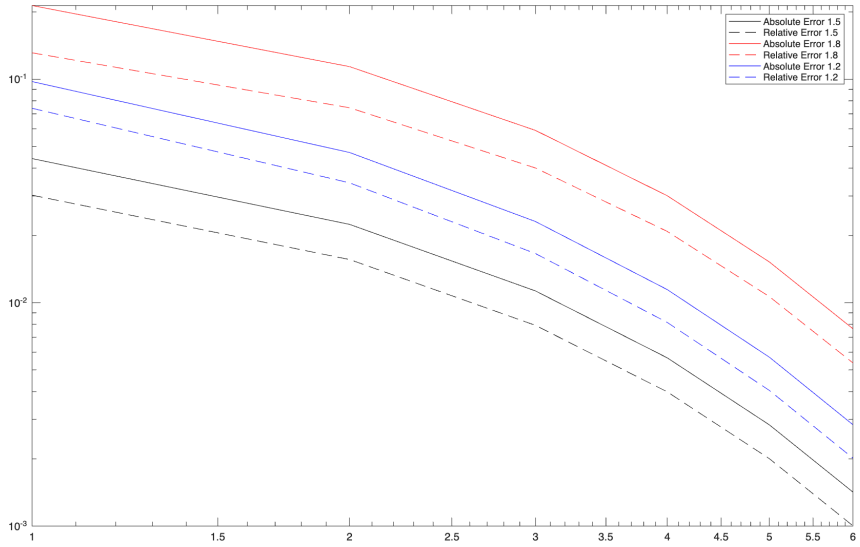

FIGURE 6. Log-Log plot of the absolute and relative error for $x_{n+1} = 1/4(3x_n + 2/x_n)$ with changes to initial conditions y_0